TESTING

COMPONENT LEVEL

Software Testing

Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.

The main objective of testing is:

To uncover errors and defects before the software is delivered to the customer.

Strategic Approach

- Technical reviews should be performed before testing.
- Testing begins at the component level.
- Testing progresses toward integration of the complete system.
- Different testing techniques are used at different stages.
- Developers are responsible for testing their components.
- An **Independent Test Group (ITG)** may be involved in larger projects.
- Testing and debugging are different activities.

V & V

- *Verification* refers to the set of tasks that ensure that software correctly implements a specific function.
- *Validation* refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements. Boehm [Boe81] states this another way:
  - *Verification:* "Are we building the product, right?"
  - *Validation:* "Are we building the right product?"

Organizing for Software Testing

- Who is going to test the system????
- The developer –it make uncover the errors they may unconsciously design tests that demonstrate that their software works rather than tests that attempt to break it.

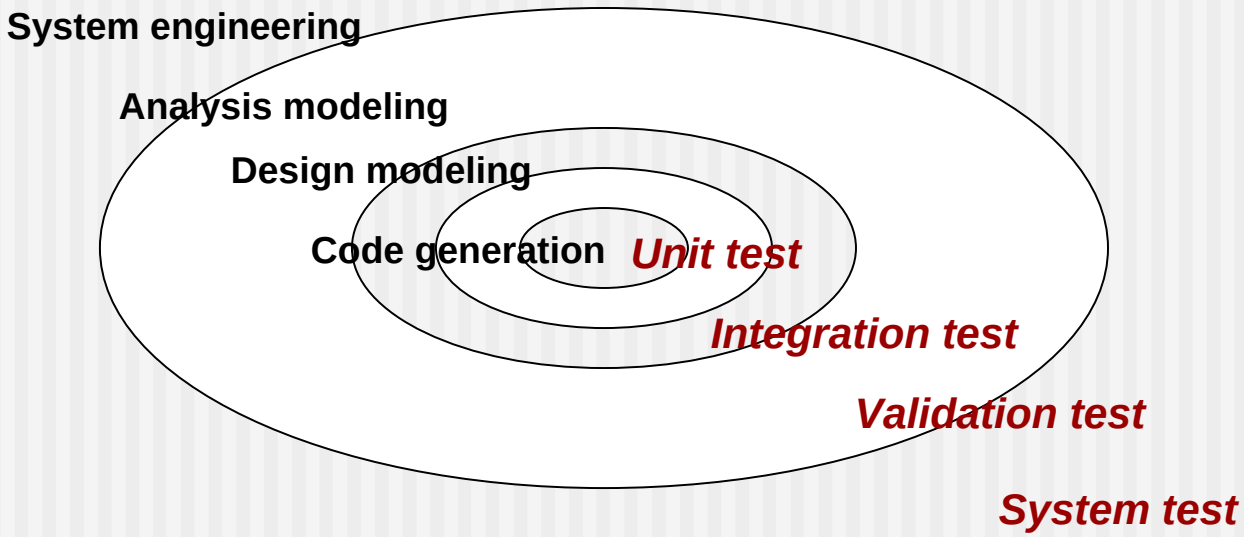
or

- An independent tester- involves during the testing periods…
- Both are included in testing period of a system

ITG

- An **Independent Test Group** provides an independent viewpoint.
- Responsibilities include:
- Designing independent tests
- Finding defects
- Evaluating software objectively
- Working with developers to resolve defects

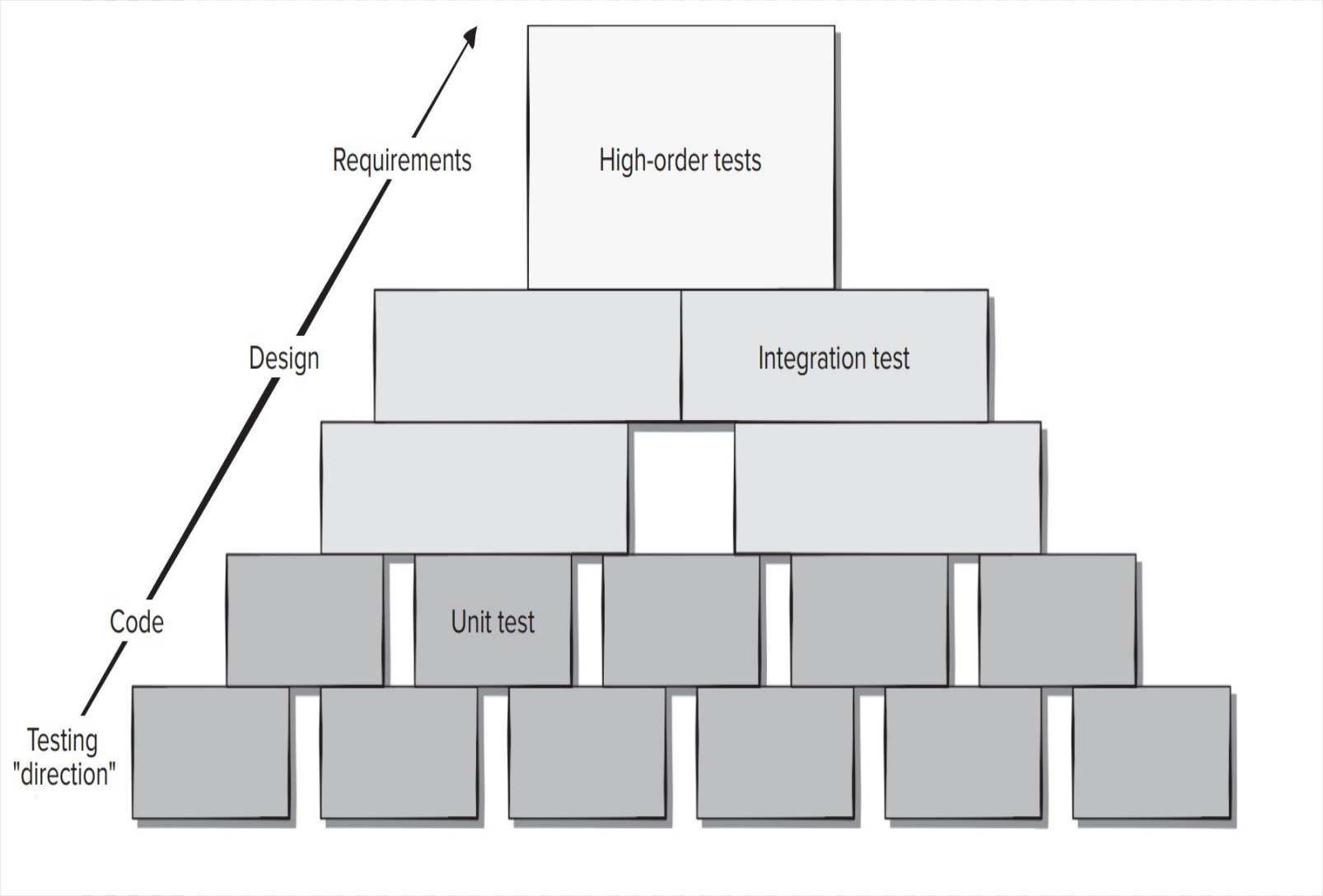
Testing Strategy



Testing Strategy

- Testing begins with small software components and gradually moves toward the complete system.

- **Requirements → Design → Code → Unit Testing → Integration Testing → Validation Testing → System Testing**
- The basic principle is:
- **Testing begins "in the small" and progresses "to the large."**

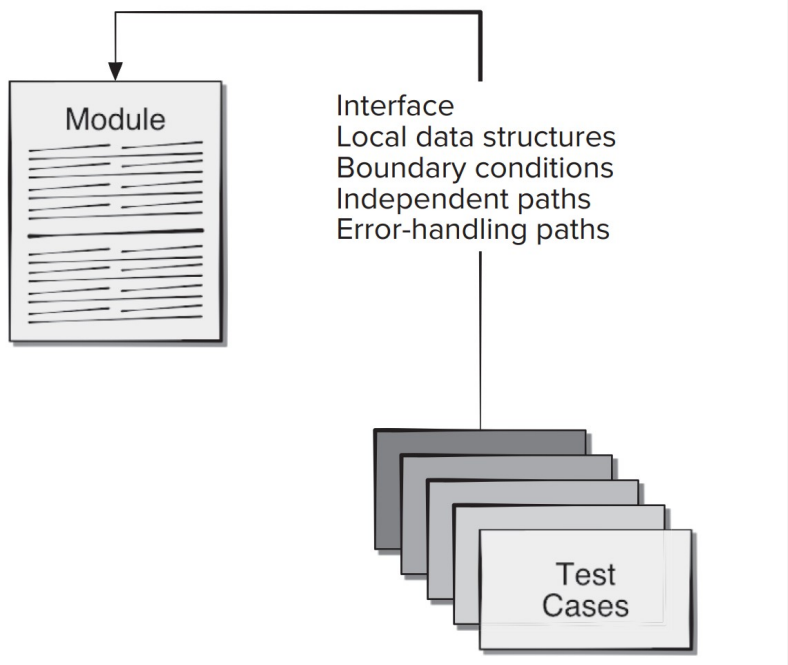

Requirements
Design
Code
Testing
"direction"
High-order tests
Integration test
Unit test

PLANNING AND RECORD KEEPING

- Plan the test activities like ,
- Deciding what to test, how to test, when to test etc
- Each activities systematically documented

Test-Case Design

- Design test cases to check weather the system works properly.
- It is a good idea to design unit test cases before you develop code for a component



Test cases contains…..

- The module interface is tested to ensure that information properly flows into and out of the program unit under test
- Local data structures are examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution.
- Data flow across a component interface is tested before any other testing is initiated
- Test cases should be designed to uncover errors due to erroneous computations, incorrect comparisons, or improper control flow

Requirements and Use Cases

- The requirements describes what the software system is expected to do or the constraints it must satisfy.
- During testing requirements are used as the basis for designing test cases.
- Use case describe how user /actor interacts with the system

Traceability

- Maintaining the clear link between requirements and the corresponding test cases, test results and defects

Strategic Issues

- Specify product requirements in a quantifiable manner long before testing commences.
- State testing objectives explicitly.
- Understand the users of the software and develop a profile for each user category.
- Develop a testing plan that emphasizes “rapid cycle testing.”
- Build “robust” software that is designed to test itself
- Use effective technical reviews as a filter prior to testing
- Conduct technical reviews to assess the test strategy and test cases themselves.
- Develop a continuous improvement approach for the testing process.

White box Testing

- White-box testing is a software testing technique in which the tester has knowledge of the internal structure, source code, logic, and control flow of the program.
- It is also called **structural testing** or **glass-box testing**.
- **White-box testing checks how the program works internally.**
- **For example,** *if a program contains an if–else condition, white-box testing designs test cases to execute both branches***.**

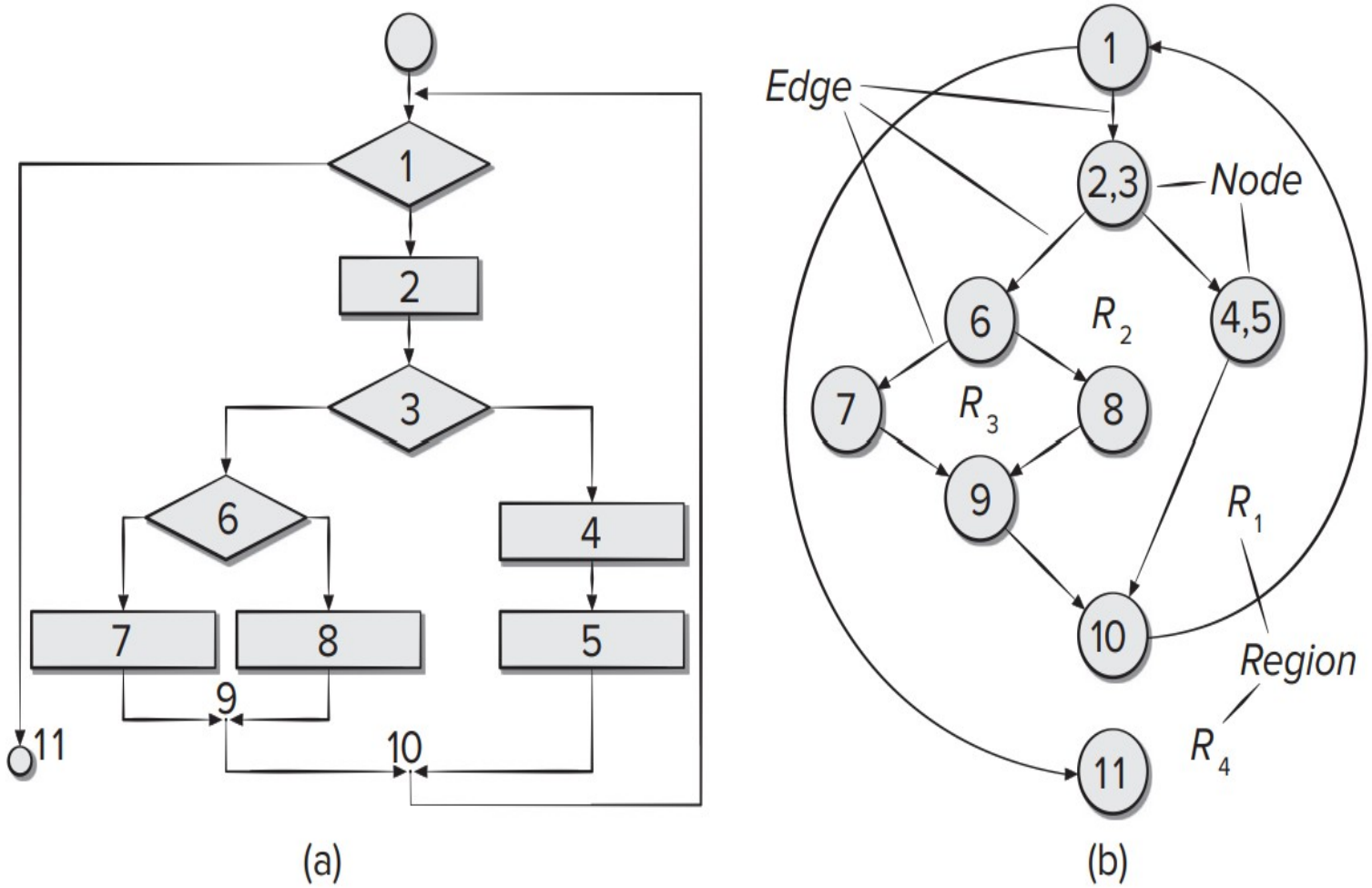
Major White-Box Testing Techniques

- **Basis Path Testing**

  Uses a control-flow graph.
- Identifies independent paths.
- Uses Cyclomatic Complexity to determine the number of independent paths.
- Design test cases so that each independent path is executed at least once.
- **Cyclomatic complexity is a software metric that provides a quantitative measure of the logical complexity of a program.**

flow graph

- A flow graph is a program graph that represents control flow of execution of a program.
- A flow graph should be drawn only when the logical structure of a component is complex.


1
2
3
6
4
7
8
5
9
11
10
(a)
Edge
1
2,3
Node
6
R_2
4,5
7
R_3
8
9
R_1
10
Region
R_4
11
(b)

- Path 1: 1-11
- Path 2: 1-2-3-4-5-10-1-11
- Path 3: 1-2-3-6-8-9-10-1-11
- Path 4: 1-2-3-6-7-9-10-1-11
- Note that each new path introduces a new edge.
- The path 1-2-3-4-5-10-1-2-3-6-8-9-10-1-11 is not considered to be an independent path because it is simply a combination of already specified paths and does not traverse any new edges.

- Complexity is computed in one of three ways:
- 1. The number of regions of the flow graph corresponds to the cyclomatic complexity.
- 2. Cyclomatic complexity V(G) for a flow graph G is defined as
- $V(G) = E - N + 2$
- where E is the number of flow graph edges and N is the number of flow graph nodes.
- 3. Cyclomatic complexity V(G) for a flow graph G is also defined as $V(G) = P + 1$
- where P is the number of predicate nodes contained in the flow graph G

- **Control Structure Testing**
- **Condition testing:** is a test-case design method that exercises the logical conditions contained in a program module.
- **Data flow testing :**selects test paths of a program according to the locations of definitions and uses of variables in the program.
- **Loop testing** is a white-box testing technique that focuses exclusively on the validity of loop constructs.

Black-Box Testing

Black-box testing, also called behavioural testing or functional testing, focuses on the functional requirements of the software.

Black-box testing attempts to find errors in the following categories:

(1)incorrect or missing functions

(2) interface errors

(3) errors in data structures or external database access

(4) behavior or performance errors

(5) initialization and termination errors.

- **Interface Testing**
- Interface testing is used to check that the program component accepts information passed to it in the proper order and data types and returns information in proper order and data format.

Equivalence Partitioning

- Equivalence partitioning is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived.
- An ideal test case single-handedly uncovers a class of errors

Boundary Value Analysis (BVA)

- focuses on testing the boundary values (edges) of valid and invalid input ranges.
- It helps identify errors that commonly occur at the limits of input conditions.
- BVA focuses on boundary values of valid and invalid input ranges.
- It tests values at the minimum, just above minimum, maximum, just below maximum, and sometimes outside the valid range.

Object oriented testing

- The testing strategy shifts from evaluating procedural functions to evaluating classes, objects, and their runtime interactions.
- Because object-oriented programming (OOP) introduces unique mechanisms like encapsulation, inheritance, and polymorphism, traditional testing methods must be adapted.

1. Class Testing (The Unit Level)

- In conventional software, the smallest testable unit is an isolated function or module. In OOT, **the encapsulated class is the smallest testable unit**.
- Class testing for OO software is focuses on the operations encapsulated by the class and the state behaviour of the class.

2. Behavioral Testing (State-Driven Testing

- Behavioral testing shifts from evaluating individual methods to validating the dynamic changes of an object over time.

Software Testing— Integration Level

Software Testing Fundamentals

- software testing is a systematic activity intended to discover errors in software and determine whether the software behaves as expected.
- The primary goal is to find errors, not simply to demonstrate that the software works correctly.
- **A good test is one that has a high probability of discovering an error.**
- Testing becomes particularly important during integration because individual modules may function correctly when tested separately but may fail when they interact with other modules.

Two general approaches to testing

- Software can be tested from two perspectives:
- **External perspective**: Check whether the software performs the functions specified in its requirements.
- **Internal perspective:** Check whether its internal logic, data structures, and interactions work correctly.
- These perspectives lead to black-box testing and white-box testing.

Black-Box Testing

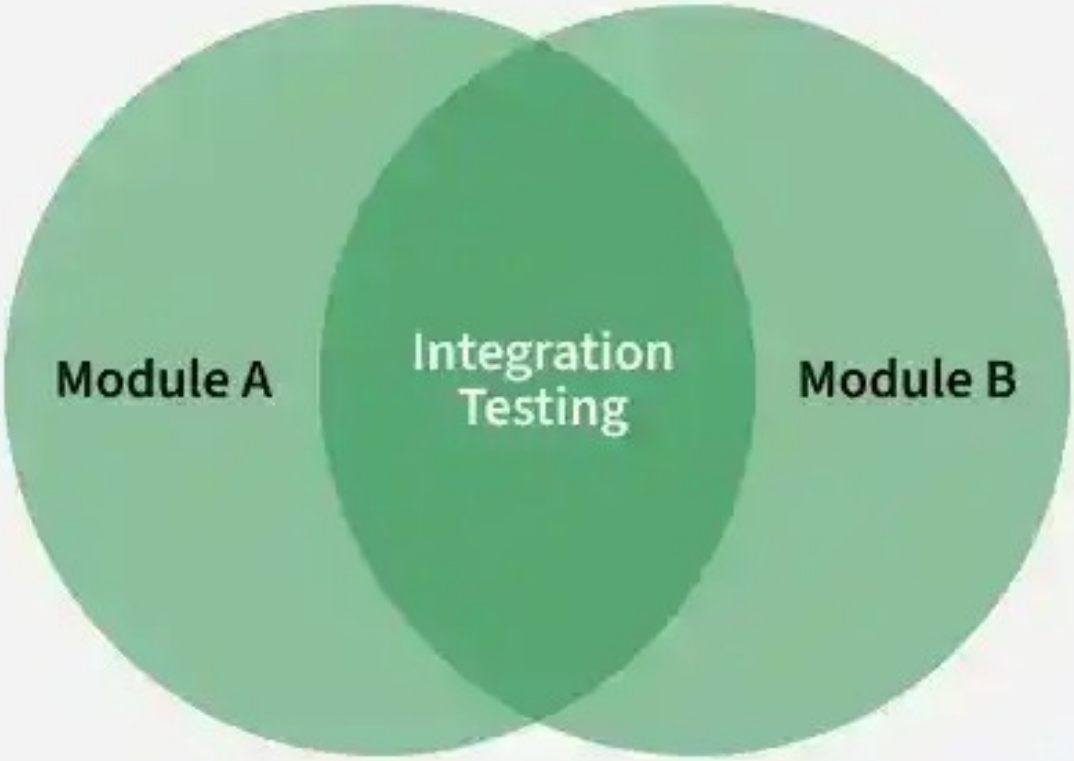
- Black-box testing examines software functionality without requiring knowledge of its internal logical implementation.
- It focuses on component interfaces, inputs, outputs, preconditions, postconditions, and required behaviour.
- **Black-box testing during integration:**
- Black-box integration testing checks whether these components collectively perform the required function, even if each component has already passed unit testing.

White-Box Testing

- White-box testing, also known as glass-box testing or structural testing, derives test cases using knowledge of the internal implementation of software.
- It examines procedural logic, control paths, data structures, and interactions among components.

Integration Testing

- A systematic technique for constructing the software architecture while testing for errors associated with the interfaces and interactions between components.
- The objective is to take individually unit-tested components and combine them into a program structure that follows the software design.


Module A
Integration Testing
Module B

Why is integration testing necessary?
Even if every component works correctly on its own, errors may occur when they are combined.
Common integration errors include:

- **Data loss across interfaces:** Information may be lost or incorrectly transferred between modules.
- **Interface incompatibility:** Components may expect different data formats or parameter types.
- **Unintended side effects**: One component may change data or state used by another component.
- **Incorrect combined functionality:** Several individually correct subfunctions may fail to produce the expected overall function.
- **Accumulated imprecision:** Small inaccuracies in separate components may combine into an unacceptable result.
- **Global data structure problems**: Components may access or modify shared data incorrectly.

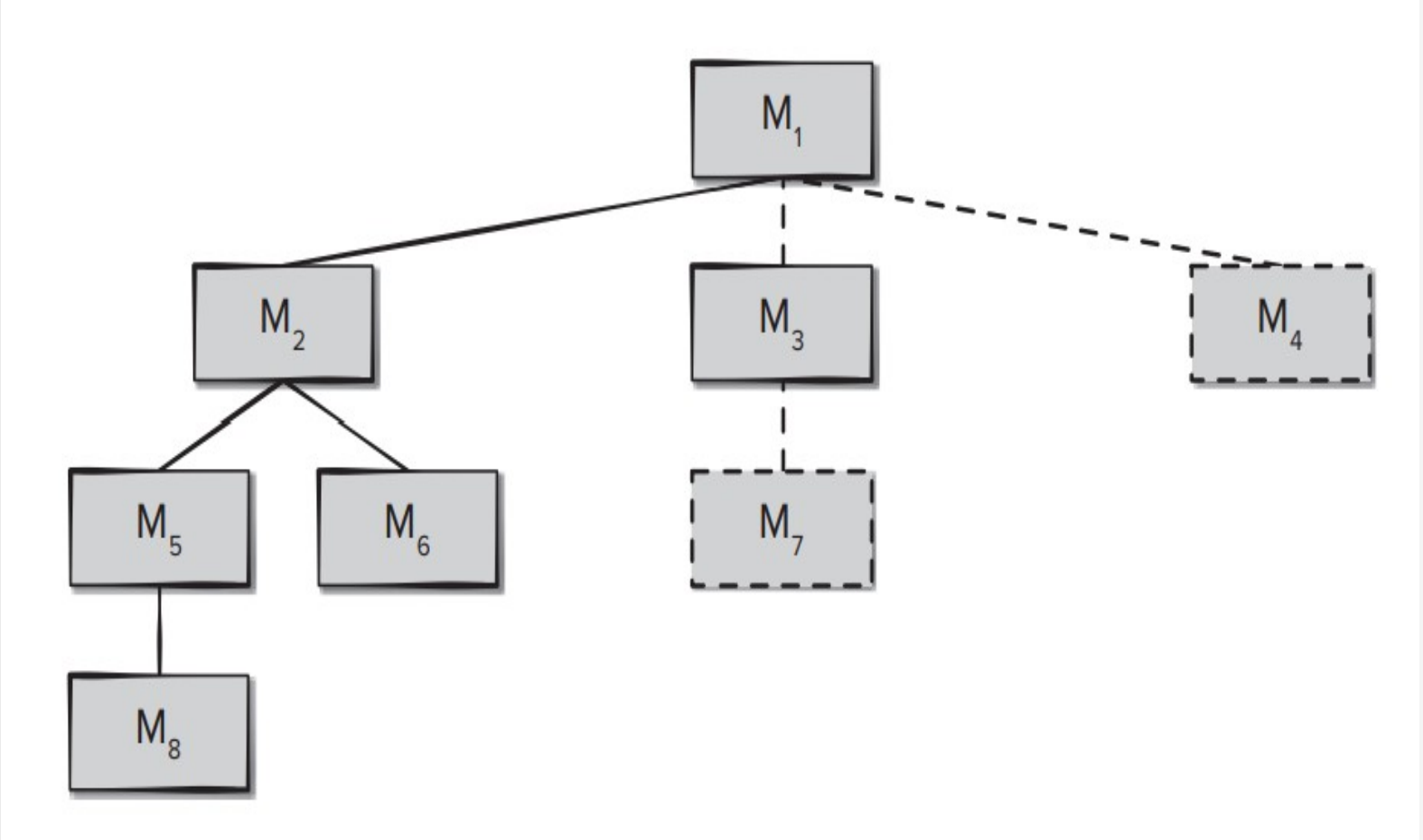
Top-Down Integration Testing

- Top-down integration testing is an **incremental strategy** in which components are integrated by moving downward through the control hierarchy, beginning with the main control module.
- **Modules** (also referred to as components)are integrated by moving downward through the control hierarchy, beginning with the **main control module** (main program). **Modules subordinate** (and ultimately subordinate) to the main control module are incorporated into the structure in either a depth-first or breadth first manner.

- Two approaches are used:
- **Depth-first integration:** Integrates all components along one major control path before moving to other paths.
- **Breadth-first integration:** Integrates components at the same hierarchical level before proceeding to the next level.

stub

- a stub is a temporary, dummy piece of code used as a placeholder for lower-level modules that are not yet developed or integrated.
- Because top-down testing starts with the highest-level architectural modules (like the main control loop or user interface) and moves downwards, the high-level code will often try to call functions or services that do not exist yet.
- A stub simulates the behavior of those missing sub-modules so testing can proceed without delays

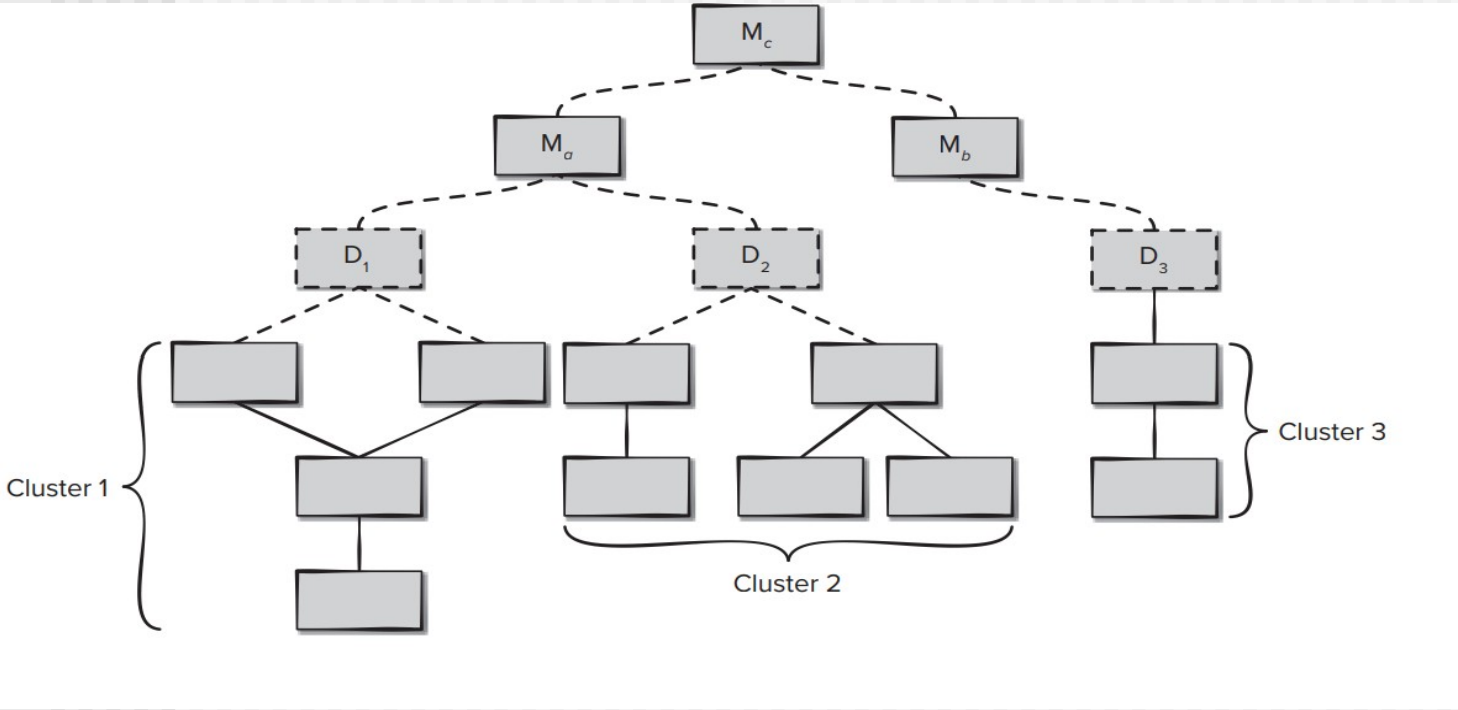

M1
M2
M3
M4
M5
M6
M7
M8

The top down integration process is performed in a series of five steps:

- 1. The main control module is used as a test driver, and stubs are substituted for all components directly subordinate to the main control module.
- 2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
- 3. Tests are conducted as each component is integrated.
- 4. On completion of each set of tests, another stub is replaced with the real component.
- 5. **Regression testing** may be conducted to ensure that new errors have not been introduced.

Bottom-Up Integration

- Begins construction and testing with atomic modules (i.e., components at the lowest levels in the program structure). Bottom-up integration eliminates the need for complex stubs



- **A bottom-up integration strategy may be implemented with the following steps:**
- 1. Low-level components are combined into clusters (sometimes called builds) that perform a specific software subfunction.
- 2. A driver (a control program for testing) is written to coordinate test-case input and output.
- 3. The cluster is tested.
- 4. Drivers are removed and clusters are combined, moving upward in the program structure.

Continuous Integration

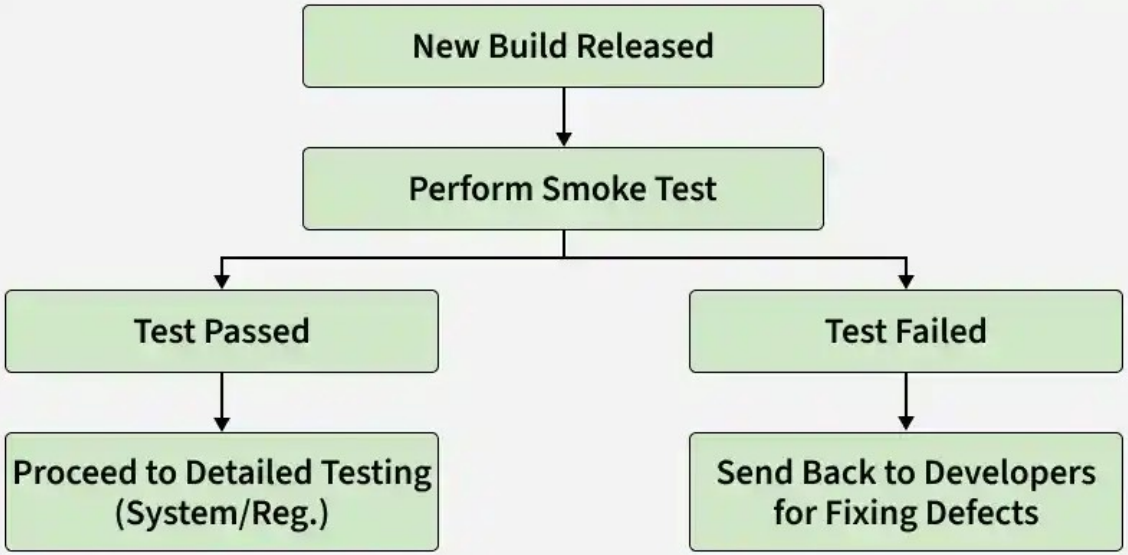
- Continuous integration is the practice of merging components into the evolving software increment once or more each day.
- This is a common practice for teams following agile development practices such as XP or DevOps

How continuous integration works

- Developers implement and test their components.
- The components are merged into a shared software build.
- Automated or repeatable tests are executed.
- Errors and incompatibilities are identified.
- Defects are corrected, and the updated build is tested again.

Smoke testing

- Technique used to verify whether the critical functionalities of a new build are working correctly.
- It ensures the build is stable enough for further detailed testing.
- Verifies the application's core functionalities.
- Ensures the build is stable for further testing.
- Helps identify major defects at an early stage.



Benefits of smoke testing

- **Reduced integration risk**: Major incompatibilities are found early.
- **Improved quality:** Functional and architectural problems can be detected during construction.
- **Simpler diagnosis**: Newly added components are likely places to investigate when a new error appears.
- **Easier progress assessment:** Regular working builds show how much of the software has been integrated successfully.
- **Smoke testing resembles regression testing, but the emphasis differs:**
- **smoke testing checks whether a build is stable enough for further testing, while regression testing checks whether changes have broken previously working behavior.**

Integration Test Work Products

- An integration test specification documents the overall integration plan and the detailed tests to be performed. It becomes part of the software configuration.
- **Test Plan**: The test plan describes the overall approach to integration testing.
- It specifies:
- Integration phases and incremental builds.
- Functional areas to be tested.
- The integration schedule.

- **Test procedure**
- The test procedure describes the specific activities required to carry out the test plan.
- It includes the order in which components are integrated and the tests conducted at each integration step.
- **Test report**
- A test report records actual test results, problems encountered, and relevant observations.
- It may be appended to the test specification so that the team can access the latest results and the current state of the software increment.

Artificial Intelligence and Regression Testing

Regression Testing

- *Regression testing* is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects
- Whenever software is corrected, some aspect of the software configuration (the program, its documentation, or the data that support it) is changed.
- Regression testing helps to ensure that changes (due to testing or for other reasons) do not introduce unintended behavior or additional errors.
- Regression testing may be conducted manually, by re-executing a subset of all test cases or using automated capture/playback tools.

Artificial intelligence in regression testing

- Using AI techniques to help select, prioritize, generate, and execute regression test cases more efficiently after software changes.

How AI can support regression testing

- **Test-case selection:** AI can identify test cases related to the modified code or affected components.
- **Test prioritization:** AI can rank test cases so that important tests are executed first.
- **Test-case generation:** AI can help generate new test cases based on code changes, requirements, or previous failures.
- **Failure prediction:** AI can analyze historical test results to identify areas that are more likely to contain defects.
- AI can help detect outdated or broken test scripts when the software interface or functionality changes.

Integration Testing in the OO Context

- **Integration testing in the object-oriented (OO) context** is the process of testing how classes and objects work together after their individual components have been tested.
- The focus is on the collaboration among classes, message passing, and interactions between objects.

Approaches to OO Integration Testing

- Two important approaches discussed in this context are:
- **Thread-based testing**
- **Use-based testing**
- Other techniques include **fault-based testing and scenario-based test-case design**, which examine possible errors and realistic user activities.

OO Testing Strategy

- class testing is the equivalent of unit testing
  - operations within the class are tested
  - the state behavior of the class is examined
- integration applied three different strategies
  - thread-based testing—integrates the set of classes required to respond to one input or event
  - use-based testing—integrates the set of classes required to respond to one use case
  - cluster testing—integrates the set of classes required to demonstrate one collaboration

Fault-Based Testing and Scenario-Based Testing

- These two techniques are used in object-oriented integration testing.
- Both help identify errors in the way classes and objects interact, but they approach testing differently.

Fault-Based Testing

- A testing technique that focuses on identifying and testing for plausible faults in an object-oriented system.
- The tester anticipates the kinds of errors that may occur and designs test cases to reveal them.
- **Works like**

1. Identify possible faults in the system.
2. Determine which operations or collaborations between classes could produce those faults.
3. Design test cases to expose the suspected errors.
4. Execute the tests and examine the results.

Scenario-Based Testing

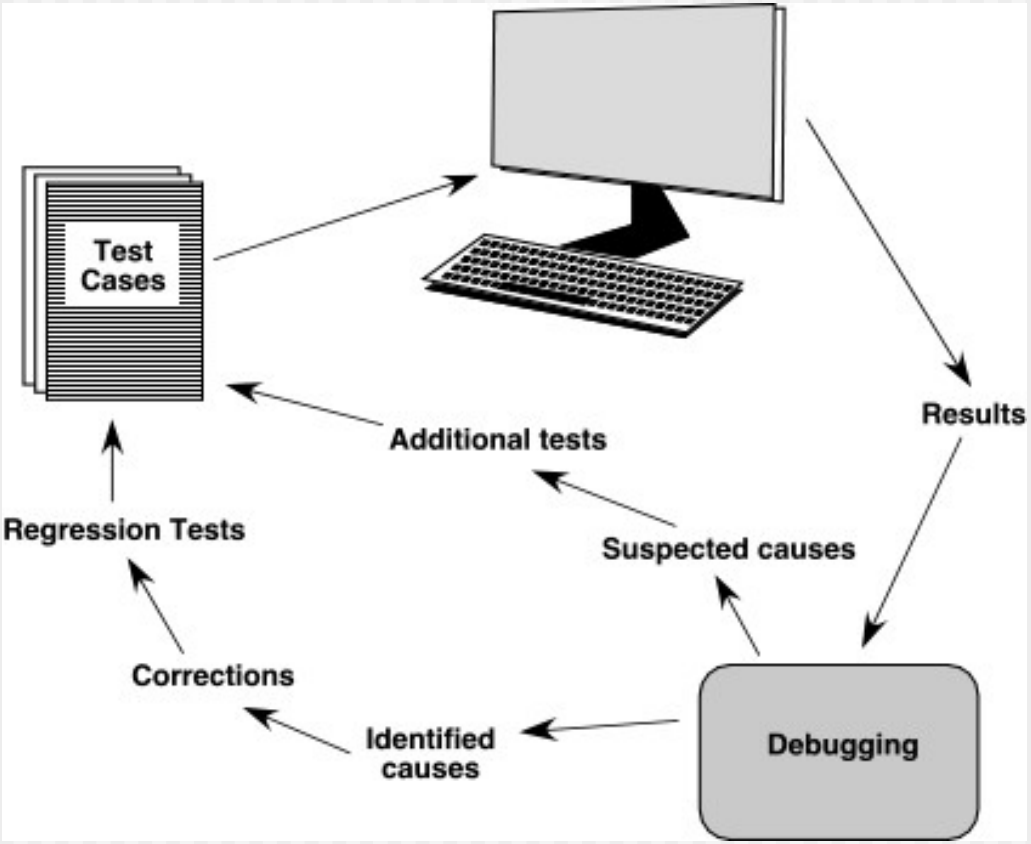
- A testing technique that derives test cases from the tasks users perform and the scenarios or use cases through which they interact with the system.
- Instead of focusing mainly on individual suspected faults, it examines how the system behaves when users perform realistic tasks involving multiple classes and subsystems.

Validation Testing

- Validation testing determines whether the completed software conforms to its requirements and functions as the customer reasonably expects.
- It begins after integration testing has assembled the software and corrected interface errors.

Debugging and Anti-bugging

The Debugging Process



Debugging

- Process of identifying, analyzing, and fixing errors, or bugs in software or hardware to restore expected performance. It involves discovering the root cause of the error and applying a fix.
- **Antibugging**
- Refers to proactive software design and coding practices that make bugs impossible or immediately obvious rather than fixing them after they occur.